

Quick Guide To Developing Training Data for Accelerometer-Based Animal Behaviour Classification

Oakleigh Wilson

April 6, 2026

Abstract

In this document, I provide a quick guide to developing accelerometer training data for a supervised machine learning classification workflow. This is mostly based on personal experience and was written for a research assistant as a general guide only.

Contents

1	Introduction	2
1.1	Some Common Sense Notes about Labelled Data	2
1.2	How Much Labelled Data Do I Need?	2
2	Aligning Accelerometer with Video	3
2.1	Overview	3
2.2	Rough Alignment	3
3	Annotating the Accelerometer Clips	4
3.1	Setting Up the Interface	4
3.2	Aligning Video and Accelerometer	4
3.3	Annotating Behaviours	5
3.4	Save	5
4	Cleaning Accelerometer Data	6
4.1	Using MATLAB to clean the labels	6
5	Complete	6

1 Introduction

1.1 Some Common Sense Notes about Labelled Data

Most of what you need to know about labelled data is common sense, but thinking through it explicitly ahead of time can save some trouble (and disappointment) down the line.

Labelled data is the information we give the machine learning model to "teach" it how to do the task we want it to do. This can take many forms, but for accelerometer-based animal behaviour classification specifically, this will be examples of what the accelerometer trace looks like for each of the behaviours we are interested in detecting. That is, a sample of the raw accelerometer readings throughout the behaviour, and a label defining what that behaviour was.

When constructing this labelled data, it is important to try to represent reality as closely as possible. Keep in mind these limitations:

- The model cannot do anything we have not explicitly taught it to do. It cannot detect behaviours we have not shown it in the training set. It will not give us performance estimates for behaviours not included in the test set. It cannot manage transitions it hasn't seen before.
- The model doesn't know what it doesn't know and will force classify **everything** it encounters into one of the known classes. For more notes on this, see my recent paper: [here](#).
- We only know how well our model performs on the data we have: if your data is very weird in some way, and we both train and test on that weird data, we may get very high performance scores, but these performance scores may not generalise to the broader data. To learn more about the consequences of wonky testing, see my other paper: [here](#).
- Be careful what you ask for: If you optimise a model for the accuracy of the majority class, it will give you a model that is accurate for the majority class (though potentially at the expense of precision of the minority class...). Include everything you can.

1.2 How Much Labelled Data Do I Need?

The short answer is: as much as possible. The long answer is that this is a very complicated question that has not yet been clearly defined in the literature, and the amount will be dependent on the variation: 1) within the behaviours you're trying to detect, 2) between the behaviours you're trying to detect, 3) within individuals, 4) between individuals. Until this can be more definitely defined, my gut tells me you benefit more from less samples, but across more events and individuals, than from a singular long stretch of the same behaviour from the same individual. That is, 2 x 1 min feeding from each of 10 individuals is more valuable than a single 20 minute stretch of uninterrupted feeding from the same individual.

2 Aligning Accelerometer with Video

2.1 Overview

In order to create precise and accurate training data, we require the video of the animal to be synchronised with the accelerometer data from the same animal. The easiest way to do this is to make sure the accelerometer and the camera are set to the same clock before releasing the animal. While this seems simple in theory, it nearly never actually happens.

There are several ways the two clocks can become desynchronised:

1. We forget to synchronise them in the first place.
2. Every device is in a different time zone (i.e., the display time may look the same, but the encoded time is different — this can be a major sneaky problem down the line).
3. The devices start off synchronised but drift due to technological imperfections.

Long story short, it is frequently the case that our videos do not match with the accelerometers. Therefore, before we can label the accelerometer trace with the behaviours from the video, we must match them up.

If your data is already synchronised, skip ahead to Section ???. Otherwise, perform the rough alignment in R first (Section 2.2).

2.2 Rough Alignment

Information You Need

- The accelerometer, with known times and time zone (e.g., UTC)
- The videos, with known times and encoded time zone (e.g., local time, UTC+2, South Africa — Africa/Johannesburg)
- The conversion between accelerometer and camera time zones
Example: camera time – 2 hours = accelerometer time (both now in UTC)

Note: It may be the case that every camera has a different setting. For example, one camera had a hardcoded daylight savings setting even when daylight savings were not in effect — check carefully.

Steps

1. Use the R script `Scripts/RoughAlignment/AccelDelayFinder.R`
2. Extract the times that the video probably occurred based on field notes (convert local time to UTC as needed)
3. Plot the probable segment using the Shiny app and open the video in another window
4. Use the slider to adjust when the video starts and ends relative to the accelerometer
5. When the alignment looks reasonable, click **Save** — this saves the exact accelerometer segment displayed in the plot, to be used in the annotation phase

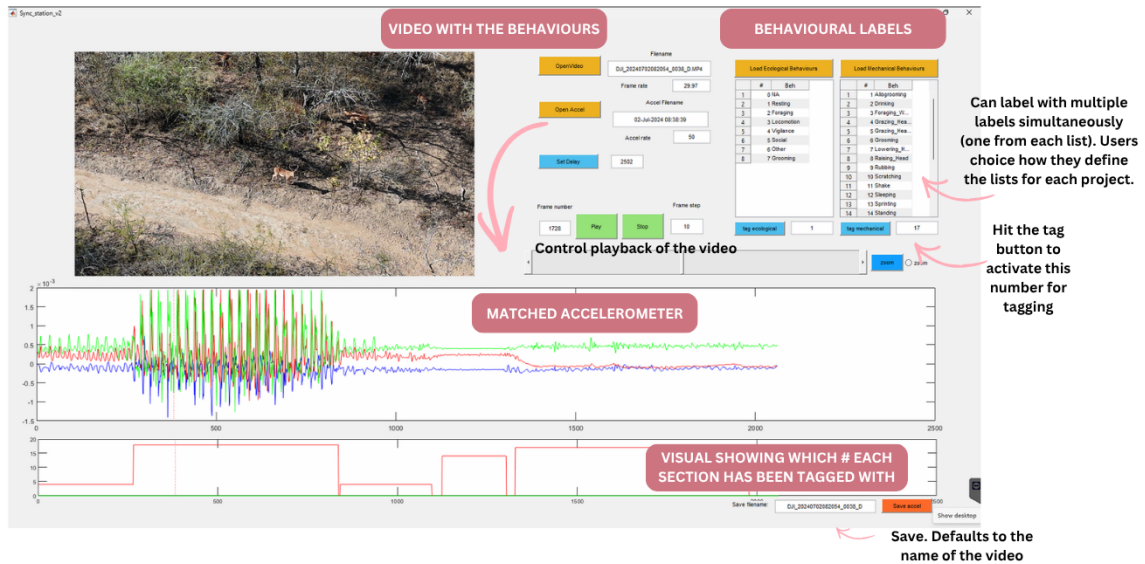


Figure 1: Using the Sync Station GUI

3 Annotating the Accelerometer Clips

3.1 Setting Up the Interface

1. Open the MATLAB interface (`Sync_station_vs.m`) Hit the Run button (looks like a play) in the Editor tab.
2. This should display something like this: 1 ... though it will be empty (no video and no accel or labels)
3. **Open the video** using the button and file explorer
 - Videos are located in `Project_Name/Raw_Data/`
 - Navigate to the animal name (e.g., `Clio`)
 - Select the correct video file (e.g., `MAH02857`)
4. **Open the accelerometer data** using the button and file explorer
 - The accelerometer file will be in the same folder as the video
 - The file should share the same name as the video
Example: video MAH02857 → accelerometer MAH02857_clipped_accel.csv
5. **Wait for everything to load** — depending on file sizes, this may take a moment
6. **Load the behaviours**
 - Locate the `Behavioural_labels.csv` file in `Project_Name/Raw_Data/`
 - Click each *Load Behaviours* button and load the associated list
 - *Ecological behaviours* — high-level, broad classifications
 - *Biomechanical behaviours* — fine-scale classifications

3.2 Aligning Video and Accelerometer

1. Find an event common to both the video and accelerometer trace (e.g., tapping the device, or an animal standing up suddenly from rest)
2. Locate the event in the accelerometer display
 - Use the *Zoom* button for a closer look
 - Left-click at the start of the region, then a short distance into the trace — the display will update to show only that window

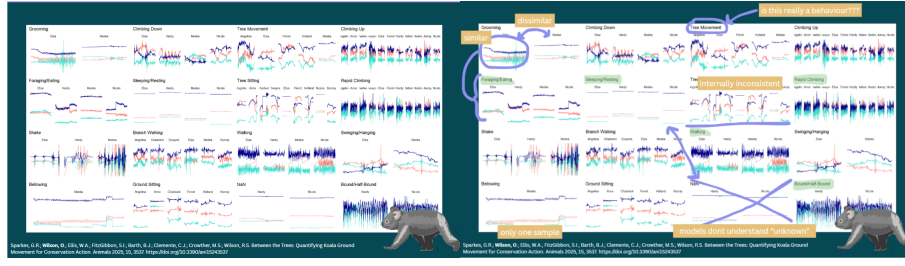
3. Adjust the delay by playing the video back and forth across the defining event, moving the vertical red line in the accelerometer display until it matches
4. Click Set Delay, then click on the accelerometer display to place the alignment line

3.3 Annotating Behaviours

1. Observe and decide on an annotation
 - Pause the video when a behaviour occurs
 - Select the best description from the menu list
 - Type the number into the box and click *Tag Behaviour*
 - Left-click the start of the behaviour, then the end
2. Repeat for each behaviour in sequence
3. Annotate every second — it is important that there are no gaps. When an unlisted event occurs, either:
 - Tag it as *transitional* or *non-behaviour*, or
 - Edit the behaviour CSV outside of MATLAB to add a new option, then reload the file in the GUI

3.4 Save

When you are happy with the annotations, save the file. The output will be written to the same folder as the clipped accelerometer file, with `_tagged` appended to the filename.



(a) Original provided labels.

(b) With quality notes.

Figure 2: Sanity checking the quality of the labelled annotations from a project on captive koalas.

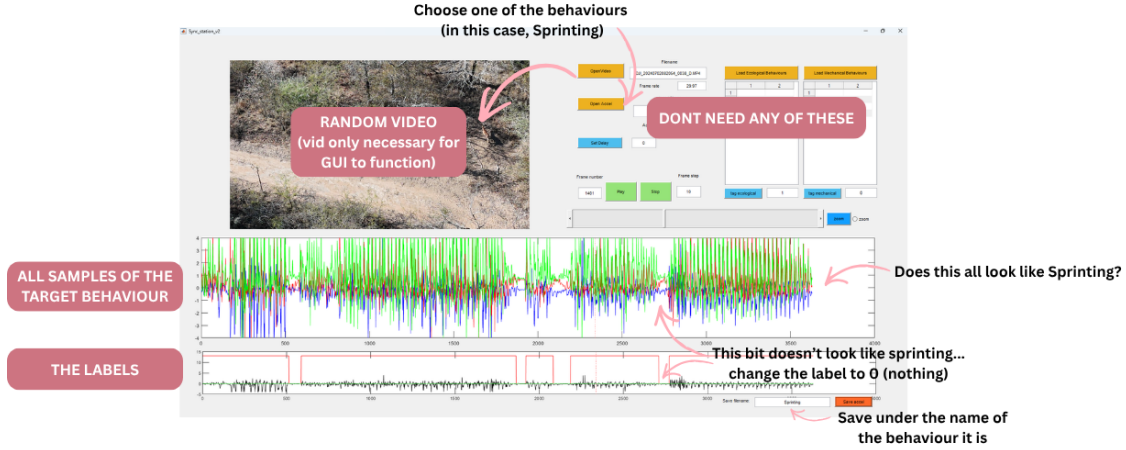


Figure 3: Cleaning any sections of the annotations that don't match.

4 Cleaning Accelerometer Data

It is a good idea to check the labelled data fairly regularly as it might not always look the way we expect it to. For an example, I have included an image of some data that was sent to me [2](#). On the left it is what I received, on the right are annotations of some issues with it.

There are some classes that are internally highly variable, some classes that look exactly the same, and some that look like nothing at all. Before we proceed with trying to get the model to sort it, we can clean it.

4.1 Using MATLAB to clean the labels

When you have annotated the data, structure it in R into behavioural categories (that is, all the examples from the same class together) and then load it back into Matlab based on the category. By putting all of the data from each activity category into one trace, we can better see whether any errors have snuck in. Clean the data by resetting the class label to 0 for anything that shouldn't be included, or otherwise relabelling where needed [3](#).

Continue through each of the labelled behaviours until they pass the sanity check.

5 Complete

When complete, put all the cleaned data back into a single file. Sort by ID and then Time. This is the data you will use to build the model.

Thank you!